

Components

The benchmark dashboard journey and the bring-your-own-dataset upload

EdgePilot AI - Team 15

Documentation set, 23 August 2026

3 documents in this volume

src/components/ - React components

Work packages: product UI dashboard, and vision benchmark.

What this folder is

Everything the user actually sees, in two groups by feature.

What happened here

Four files were removed from this level: `ErrorBoundary.tsx`, `Loading.tsx`, `LoadingStates.tsx` and `providers/`. Three of those were overlapping answers to the same question, and the App Router already has a convention for it - `loading.tsx` and `error.tsx` beside the route. What genuinely needed sharing is 85 lines in `dashboard/StateViews.tsx`.

`vision/` is new, and it holds a component with an unusual constraint: it must work entirely in the browser, because the images it handles are not allowed to reach the server.

Subfolders

Subfolder	What it holds
dashboard/	11 files - the benchmark journey
vision/	The bring-your-own-dataset upload

Connected folders

- `src/app/` - the routes that mount these.
- `docs/dashboard/` - the module guide.
- `tests/dashboard/` - the helper tests.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/`

src/components/dashboard/ - the benchmark journey

Work package: product UI and benchmark dashboard / integration lead.

What this folder is

Eleven files implementing the four-step user journey: workload, provider and model, run, results. This is where the 692 lines that used to be in `app/dashboard/page.tsx` now live, and the move is what makes them testable.

What happened here

The largest change in the folder is a **deletion**. `SetupPanel.tsx` went from 322 lines to 201 - 379 lines differing - because the UUID field and everything around it went away. Users were being asked to paste a workload uuid by hand, because the endpoint that should have created one did not persist. Once `POST /api/v1/workloads` wrote real rows, the field, the placeholder-id detection and the "this was not saved" warning all became dead weight.

The identity that replaced it is `session.ts`, and the important property is that it is **never rendered**. A user should no more be asked to see their own row-ownership key than to type their own cookie.

`InstalledModels.tsx` and `VisionHandoff.tsx` are new. The installed-models panel is always visible and filters by step: on step 1 models that cannot serve the chosen task are greyed out; from step 2 only models that will work are offered at all.

The input and output format fields became read-only derived text - an informant, not a control. They describe what the chosen task sends and returns; they were never things a user could meaningfully change.

Files

File	What it is
<code>DashboardApp.tsx</code>	204 lines. The step machine and shared state
<code>SetupPanel.tsx</code>	202 lines. Step 1 - the workload. No uuid field any more
<code>ProviderPanel.tsx</code>	295 lines. Step 2 - the live catalogue. Unconfigured providers are visible but not selectable
<code>RunPanel.tsx</code>	316 lines. Step 3 - prompt, iterations, elapsed feedback. Tells the user their iteration count will be raised by one for the discarded cold start
<code>RunResults.tsx</code>	414 lines. Step 4 - per-iteration table with provenance badges, null-honest summary tiles, the fallback chain, readiness and its stored breakdown, evidence, assumptions, limitations, JSON export, and the cold-start row
<code>InstalledModels.tsx</code>	245 lines. The always-visible model panel, with <code>fitFor</code> deciding what is offered at each step
<code>VisionHandoff.tsx</code>	88 lines. The route into the vision dashboard
<code>StateViews.tsx</code>	85 lines. <code>EmptyState</code> , <code>ErrorState</code> , <code>LoadingState</code> - what replaced three separate top-level components
<code>api.ts</code>	243 lines. The one typed gateway. Every request goes through it, so components render exactly one error shape. Types come from the benchmark module's DTOs via <code>import type</code> , erased at build time, so the dashboard cannot drift from the measurement envelope without a compile error
<code>format.ts</code>	86 lines. Display helpers whose job is that no component ever turns a null into a 0

<code>session.ts</code>	92 lines. The browser's own durable id in <code>localStorage</code> , sent as <code>x-edgepilot-session</code> , never displayed. <code>localStorage</code> and not <code>sessionStorage</code> , because a workload registered in one tab has to still be yours in the next tab, and tomorrow
-------------------------	--

Connected folders

- `src/app/dashboard/` - the route and the `.epd-scoped` stylesheet.
- `src/app/api/v1/` - what `api.ts` talks to.
- `src/lib/` - `sessionOwner.ts`, the server half of `session.ts`.
- `src/modules/benchmark/application/dtos/` - the types `api.ts` imports.
- `tests/dashboard/` - `format.test.ts`.
- `docs/dashboard/` - the module guide, including the requests this folder made of the other work packages.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/local-runtime/`
- `src/app/api/v1/providers/`
- `src/app/api/v1/readiness/[id]/` (FOLDER.md)
- `src/app/api/v1/session-log/`
- `src/app/api/v1/workloads/`
- `src/app/vision-benchmark/`
- `src/components/`
- `src/core/logging/`

src/components/vision/ - bring your own dataset

Work package: vision benchmark.

What this folder is

One component, and one strict rule: **the images never leave the machine.**

What happened here

New. A user picks a folder laid out one directory per class (`webkitdirectory`), and the component measures a real model against their own photographs without a single image reaching the EdgePilot server. Resizing and hashing happen in the tab; classification goes from the tab straight to the user's own Ollama on localhost.

That is not a convenience decision. Benchmarking confidential data is this project's stated non-goal, and sending images to a server to be resized would have contradicted the warning the rest of the application gives.

Four things had to be got right, and three of them were got wrong first:

- **labelOf requires three path parts.** With two, a loose file sitting at the chosen root adopted the root folder's own name as its class.
- **The prompt comes from the dataset.** A deliberate failure run proved it: with the built-in seven-label prompt and a custom two-class dataset, the result was 0% accuracy and 100% invalid output. `buildVisionPrompt(labels)` builds the closed label set from whatever the user brought. Re-run properly with `red_panel/green_panel: 100%`, passed.
- **The metrics schema had to be widened.** `perClass` was pinned to exactly seven entries, so a different class count ran to completion and *then* failed validation.
- **Four declarations block the run** - SPDX licence text plus three privacy confirmations. A dataset with no stated licence and no confirmation of what is in the images is not one anybody should publish a metric from.

Files

File	What it is
<code>DatasetUpload.tsx</code>	403 lines. Folder-per-class picker, <code>labelOf</code> , the four blocking declarations, prompt construction from the dataset's own labels, and evidence JSON download

Connected folders

- `src/modules/vision-benchmark/infrastructure/` - `browser-image-processor.ts` and `browser-ollama-provider.ts`, the two adapters that exist so this can work in the tab.
- `src/app/vision-benchmark/` - the page that mounts it.
- `src/modules/vision-benchmark/core/` - `buildVisionPrompt` and the widened schemas.
- `datasets/vision-benchmark/` - the reference dataset this generalises away from.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/vision-benchmarks/`

- src/components/
- src/modules/vision-benchmark/